ERA: Evolving Reasoning Agents

Filippo Adami, Alessandro De Vidi, Edoardo Di Tommaso, and Carlo Zamuner

***Abstract*—Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language processing, yet constructing effective reasoning chains (Chain-of-Thought) often requires manual, trial-and-error prompt engineering. Our work introduces Evolving Reasoning Agents (ERA), a novel framework that applies neuroevolutionary principles to automate the discovery of cognitive architectures. By treating prompts and reasoning steps as genes within a topological genome, ERA evolves agents capable of complex multi-step reasoning without human intervention. We describe the separation of Genotype and Phenotype, the mutation operators adapted for semantic space, and the fitness evaluation mechanisms.**

***Index Terms*—Evolutionary Algorithms, Large Language Models, Neuroevolution, Prompt Engineering, Cognitive Architectures.**

I. Introduction

THE advent of Large Language Models (LLMs) has shifted the paradigm of Artificial Intelligence from model training to prompt engineering and context management. While standard prompting yields impressive results, complex tasks often require "Chain-of-Thought" (CoT) reasoning, where the model breaks down problems into intermediate steps. However, designing these chains is a fragile, manual process, often specific to a single model or domain.

We propose a shift from manually engineering static prompts to evolving dynamic reasoning architectures. Drawing inspiration from biological evolution, we introduce the Evolving Reasoning Agents (ERA) project. The acronym ERA reflects our ambition to define a new era in automated reasoning, in which agents adapt their internal cognitive topology to the task at hand.

In this work, we translate concepts from Neuroevolution of Augmenting Topologies (NEAT) into the semantic domain of LLMs. Instead of neurons and weights, our agents evolve textual instructions and logical connections.

II. Methodology: The ERA Architecture

The ERA framework is built upon a strict separation of the evolutionary blueprint (Genotype-Gene) and the executable instance (Phenotype-Trait). This separation allows for efficient mutation and crossover operations while keeping evolution logic separate from LLM's runtime execution.

A. Genotype: The Agent Genome

The core of ERA is the *AgentGenome*, a directed graph $G = (V, E)$ adapted from NEAT [2] for semantic operations. **PromptNodes** replace activation functions with natural language *Instructions*, *Embeddings* for genetic distance, and *Innovation Numbers*. **Connections** represent unweighted logical flow ($u \to v$) modulated by LLM attention, each tracking an *Innovation Hash* and *Enabled Bit*.

To support CoT, the genome is structured as a DAG anchored by Start/End nodes. We enforce acyclicity to prevent context loops and employ a modified **Kahn's Algorithm** to compile a linearized execution plan P. This ensures parent outputs are concatenated into the child's context window only after all dependencies are resolved.

B. Phenotype

While the Phenotype is responsible for executing a genome, the Phenotype handles *how* it runs: invoking the LLM for each node and passing outputs along the chain.

1) Execution Flow: Given a genome with nodes $[n_1, n_2, \ldots, n_k]$ connected in a specific topology, the Phenotype performs the following operations:

1) **Trait Encapsulation:** It wraps each genomic node into a `Trait`. A Trait combines the static instruction from the genome with the dynamic context of the current problem instance.
2) **Sequential Execution:** The Phenotype resolves the topological sort of the genome and executes traits sequentially. The output of a predecessor node ($output_i$) serves as the input context for the successor node ($input_{i+1}$).
3) **Result Aggregation:** Upon completion, the Phenotype collects all intermediate outputs and the final conclusion, returning a structured trace of the reasoning process.

2) LLM Integration: To maintain computational efficiency, the LLM is not instantiated per-phenotype. Instead, a single LLM instance (which may interface with a remote server or a local quantized model) is passed by reference to all phenotypes in the population. This is wrapped in an `LLMClient` interface that provides:

- Abstraction over synchronous and asynchronous calls.
- Support for Local HuggingFace models for lower latency.
- Mock responses for architectural unit testing.

3) Logging and Debugging: The non-deterministic nature of LLMs makes debugging evolved structures challenging. The Phenotype enforces strict logging of all LLM transactions (system prompt, user input, model response, and latency). This trace is essential for analyzing the "cognitive path" taken by the agent and determining why specific mutations yield higher fitness.

C. Mutation Operators

Evolutionary diversity in ERA is driven by a stochastic mutation engine that operates on two distinct levels: *Architectural Mutations*, which modify the global topology of the reasoning graph, and *Semantic Mutations*, which modify the internal textual instructions of individual nodes.

The authors are with the University of Trento, Trento, Italy.
Manuscript received January 4, 2026.

1) Architectural Mutations: These operators dictate the structural complexity of the agent. While derived from standard NEAT operations, they include additional safeguards to maintain the Directed Acyclic Graph (DAG) property required for logical reasoning. These mutations are quite standard and include "Add-Connection", "Remove-Connection", "Add-Node" and "Remove-Node"

2) Semantic Mutations: Unlike biological neural networks where "content" is a numerical weight, ERA nodes contain semantic instructions. We introduce a set of LLM-driven operators to evolve these prompts:

- **Expand:** The LLM rewrites the instruction to be more detailed, breaking down high-level goals into actionable sub-steps within the same node.
- **Simplify:** The LLM abstracts the instruction, removing implementation details to encourage generalization.
- **Reformulate:** The LLM paraphrases the instruction to alter the embedding space position without changing the logical intent, effectively performing a "local search" in the semantic landscape.

3) The Novel Split Mutation: Standard NEAT does not support the concept of splitting a single neuron into two sequential peers. However, in CoT reasoning, a single prompt often becomes too complex and must be decomposed into two atomic steps to improve performance. We introduce the **Gene Split** mutation.

Given a target node N with instruction I, the LLM is prompted to decompose I into two sequential steps, I_a and I_b. The graph G is then transformed into G' by replacing N with two nodes N_a and N_b, connected by a new edge $N_a \to N_b$.

Topological Validity and Proof of Acyclicity: A critical requirement for the Split mutation is that it must not introduce cycles into the DAG. We generalize this transformation for the n-to-m case (where node N has n incoming edges and m outgoing edges).

Theorem: If a graph G is a DAG, the graph G' resulting from a node split is also a DAG.

Proof Sketch: Let $Cycles(G)$ be the set of cycles in G. We assume $Cycles(G) = \emptyset$. The transformation rewires all incoming edges $u_i \to N$ to $u_i \to N_a$ and all outgoing edges $N \to v_j$ to $N_b \to v_j$. A new cycle in G' must involve the newly created edge $N_a \to N_b$. For such a cycle to exist, there must be a path from N_b back to N_a through the rest of the graph. Since N_b inherits the outputs of N, and N_a inherits the inputs of N, a path $N_b \rightsquigarrow N_a$ in G' implies the existence of a path from the outputs of N to the inputs of N in the original graph G. This would imply $N \to v \rightsquigarrow u \to N$, constituting a cycle in G. Since G is acyclic by definition, no such path exists. Therefore, the introduction of $N_a \to N_b$ cannot create a cycle.

This proof ensures that the Split operator is structurally sound regardless of the node's connectivity, allowing the system to safely increase reasoning depth.

D. Crossover

Following NEAT's approach, crossover is implemented by taking advantage of the innovation number present in all genes of a genome: given two parents, the nodes and edges are aligned thanks to the innovation number; then, a gene only present in one of the two parents is always inherited, while if present in both, it randomly inherits the gene from only one of the two. At the end of the process, a loop-prevention check procedure is run, to avoid the creation of loops from the merging of the two parents' structures.

E. Speciation

Like NEAT, species are implemented, to restrict crossover to similar individuals, and protect newborn species: since edges hold no weight, the formula to compute the distance between two individuals in edited as follows:

$$\delta = c_1 \left(\frac{E}{N}\right) + c_2 \left(\frac{D}{N}\right) + c_3 \left(\frac{E_{diff}}{N_e}\right) + c_4 \cdot \Delta\overline{W} \quad (1)$$

Where:

- E is the number of excess genes.
- D is the number of disjoint genes.
- E_{diff} is the count of different edges between the two individuals.
- N is the maximum number of nodes between the two genomes (normalized to 1 if empty).
- N_e is the maximum number of active edges between the two genomes (normalized to 1 if empty).
- $\Delta\overline{W}$ is the average embedding difference of matching genes.
- c_1, c_2, c_3, c_4 are constants that adjust the importance of each factor.

F. Adjusted Offspring Allocation

To maintain a stable population while encouraging innovation, the number of offspring n allocated to a species is determined by its performance relative to the global average. The base offspring count n_{base} is calculated as:

$$n_{base} = \begin{cases} \frac{\bar{f}_{global}}{F_k} & \text{if } \bar{f}_{global}\text{is defined} \\ |S_k| & \text{otherwise} \end{cases} \quad (2)$$

Where $\bar{f}_{global}$ is the average fitness of the entire population, F_k is the cumulative fitness of species k, and $|S_k|$ represents the current number of members in that species.

To protect structural innovation, we define a dynamic protection limit P_L that scales with the structural complexity of the genomes:

$$P_L = \gamma_{base} + (\alpha \cdot \bar{C}_k) \quad (3)$$

Here, $\bar{C}_k$ is the average complexity (total nodes and connections) of the species, γ_{base} is the base protection period, and α is a scaling coefficient (set to 0.5 in this implementation).

If the species age A_k is less than the protection limit ($A_k < P_L$), the offspring count is boosted by a protection rate r_p to prevent premature extinction. The final offspring count n_{final} is defined as:

$$n_{final} = \begin{cases} \max(1, \lfloor n_{base} \cdot r_p \rfloor) & \text{if } A_k < P_L \\ \lfloor n_{base} \rfloor & \text{otherwise} \end{cases} \quad (4)$$

This mechanism ensures that species are granted an incubation period proportional to their complexity. This allows multiple structural configurations to be explored; if a species fails to achieve competitive results after this period, it is naturally extinguished.

III. Evolutionary Manager

The Evolutionary Manager acts as the orchestrator of the ERA system. It maintains the population, handles the epoch cycles, and ensures genetic diversity.

A. Fitness Evaluation

The quality of a reasoning agent is multidimensional. We implemented a **Fitness function** F that balances correctness with structural efficiency, it is defined as a weighted sum of four components:

$$F = w_{acc} \cdot S_{acc} + w_{rat} \cdot S_{rat} - w_{tok} \cdot C_{tok} - w_{cpx} \cdot C_{cyc} \quad (5)$$

In our experiments, we set $w_{acc} = 2.0$ to prioritize functional correctness. The penalty terms are kept lightweight ($w_{tok} = 0.001$, $w_{cpx} = 0.07$) to function as a regularizer rather than a primary driver, effectively pruning "bloat" only when accuracy is comparable. The components are:

- **Accuracy (S_{acc}):** Semantic similarity (cosine) between the generated answer and the ground truth.
- **Rationale (S_{rat}):** Captures the quality of the intermediate reasoning chain.
- **Token Cost (C_{tok}):** A penalty for verbosity, encouraging concise reasoning.
- **Cyclomatic Complexity (C_{cyc}):** To prevent "spaghetti code" topologies, we calculate the cyclomatic complexity ($E - N + 2$).

This configuration aims to favor interpretable agent topologies that optimize accuracy without bloating the prompt graph.

B. Selection and Replacement Strategies

ERA employs a modular `GenerationManager`. For parent selection, we utilize **Tournament Selection** with a tournament size of $k = 7$. This relatively high selection pressure favors exploitation of high-performing traits.

To balance this greedy selection, we implemented a hybrid parent sourcing mechanism: 20% of parents are injected directly from a **Species-Level Hall of Fame**, while the remaining 80% are selected from the current generation. This transforms the Hall of Fame from a passive archive into an active *genetic reservoir*, ensuring that successful structural motifs are preserved and re-introduced even if they temporarily fall out of favor.

For survivor selection, we adopt a $(\mu + \lambda)$ strategy (Comma Plus), where the best individuals from the combined pool of parents and offspring progress to the next generation.

IV. Experiments

To validate the ERA framework, we designed an experimental environment aimed at observing the emergence of complex reasoning patterns from a minimal baseline.

A. Experimental Setup

1) Dataset Selection: To evaluate the effectiveness of our evolutionary prompt optimization, we required a benchmark that challenges the reasoning capabilities of Small Language Models (SLMs) like Gemma 3 1B without relying on closed-form multi-choice questions.

a) Selection Criteria: Our selection was driven by three requirements:

1) **Generative Nature:** The model must generate the answer from scratch. This prevents the evolutionary process from overfitting to simple probability distribution hacks common in multiple-choice tasks.
2) **Logical Complexity:** The task must require multi-step reasoning (Chain-of-Thought) to be solved, providing a gradient for the fitness function.
3) **Calibrated Difficulty:** The baseline performance of the 1B model needed to be low enough to allow improvement, but not random.

b) Benchmark Analysis: We conducted preliminary probing using *CoT-Collection* (KAIST-AI), but found it too heterogeneous for stable evolutionary convergence in early generations.

c) Selected Architecture: CLUTRR: We ultimately selected **CLUTRR** (Compositional Language Understanding and Text-based Relational Reasoning). This dataset presents semi-synthetic narratives about family relationships and queries the relationship between two specific characters (e.g., *"Alice is Bob's mother... What is Alice to Charlie?"*).

This task is ideal for ERA because:

- It depends purely on logical deduction, not external knowledge.
- The relationship graph must be traversed step-by-step, making it highly sensitive to the quality of the reasoning topology.
- The output space is constrained (kinship terms) but generative, facilitating robust fitness evaluation.

2) Model Selection: We selected **Gemma 3 1B** (1 billion parameters) as the core Large Language Model for both the phenotype execution and the breeder mutation engine. This choice was driven by two key factors:

1) **Performance Gap:** Using a weaker base model provides a significant margin for improvement, allowing us to attribute performance gains strictly to the evolved topology rather than the model's inherent zero-shot capabilities.
2) **Computational Efficiency:** The evolutionary process requires thousands of inference calls per epoch. The 1B parameter size allows for rapid generation cycles on consumer-grade hardware, enabling a high volume of evaluations within a reasonable timeframe.

3) Initialization Strategy: The "Clean Slate": To avoid the introduction of bias during the experiment, we adopted a "Clean Slate" initialization strategy, by initializing the whole Generation 0 with **30 identical individuals** composed of a single simple prompt.

This approach ensures that any structural complexity or semantic refinement observed in later generations is exclusively

the result of the ERA evolutionary process, thereby proving the system's ability to autonomously discover cognitive architectures.

B. Results

A primary limitation encountered during experimentation was high computational resource demand, resulting in excessively long runtime durations (approximately 15 generations per hour). Due to this constraint, comprehensive hyperparameter tuning was not feasible; consequently, the system has not achieved meaningful results. Nevertheless, the system demonstrated a gradual evolution of individuals toward a combination of improved fitness and increased complexity.

Additionally, the restricted batch size ($N = 10$) introduced significant variability, causing the fitness function to fluctuate substantially based on the average difficulty of the problem pool. Increasing the batch size further was precluded by the prohibitive computational time required to evaluate each individual across a larger set of problems. Despite the computational constraints limiting the evolution to approximately 15 generations per hour, the experimental data reveals a clear upward trend in the population's structural adaptation. In the early stages (Fig. 1), the population remains clustered around the baseline complexity. However, by the 66th generation (Fig. 3), we observe the emergence of a "Pareto front" where individuals balance increased reasoning depth with fitness gains.

A qualitative analysis of the evolved genomes indicates that the *Gene Split* mutation successfully decomposed monolithic prompts into specialized functional units—typically separating evidence extraction from logical deduction. The observed fluctuations in average fitness are primarily attributed to the high variance inherent in the CLUTRR dataset when evaluated with small batch sizes ($N = 10$). Nevertheless, the "Species-Level Hall of Fame" acted as a critical genetic reservoir, allowing the population to recover high-performing structural motifs even after stochastic drops in generation-wide performance.

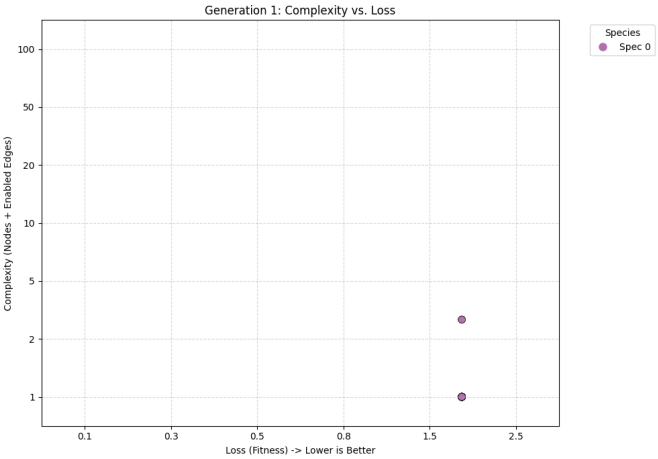


Fig. 1: 2nd generation population distribution

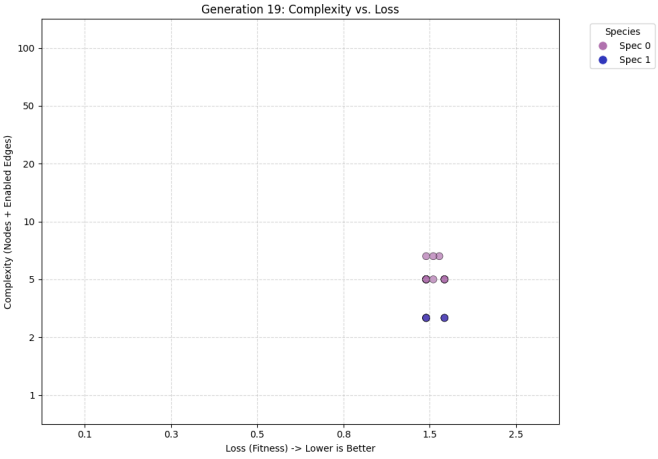


Fig. 2: 19th generation population distribution

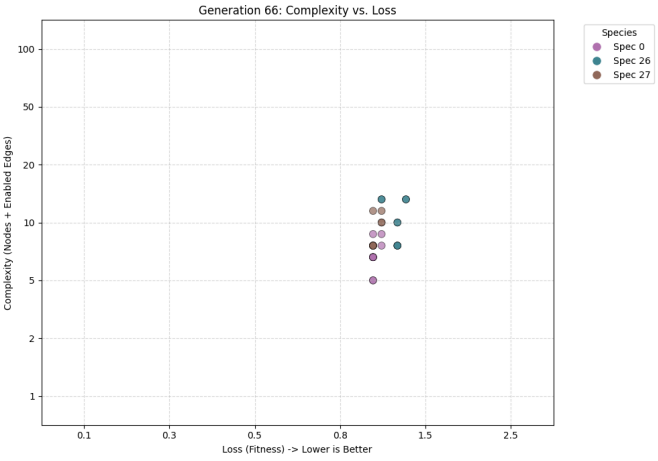


Fig. 3: 66th generation population distribution

V. Conclusion

In this work, we introduced ERA, a framework that bridges neuro-evolutionary principles with Large Language Model orchestration. Our experiments demonstrate that, given the enormous amount of variables that affect the results, and given the difficulty of the computations required, we cannot state for sure whether the approach is flawed or not. Hardware and time constraints precluded reaching a global optimum or exhaustive hyperparameter tuning. Any possible future work should focus on either making each generation run more efficient, or adopting more computational resources, to allow for runs with a more exhaustive problem pool.

References

[1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models, 2022.

[2] K. O. Stanley and R. Miikkulainen, "Evolving neural networks through augmenting topologies," *Evolutionary computation*, 2002.